Lab Practical 4 (Windows)

To develop a MapReduce program to find the grades of students based on their marks.

Run this ONLY ONCE when setting up Hadoop for the first time!

Command:

hdfs namenode –format

Starting Hadoop Services (Every Time)

Always open CMD as Administrator before running these commands!

Step 1: Start HDFS

Command:

start-dfs.cmd

Step 2: Start YARN

Command:

start-yarn.cmd

Step 3: Exit Safe Mode

Command:

hdfs dfsadmin -safemode leave

Step 4: Check Running Services

jps

Practical 4

Step 1: Create Working Directory

Commands:

```
mkdir C:\grades
cd C:\grades
```

Step 2: Create student.txt (Input File)

Command:

```
notepad student.txt
```

Enter the following student data (format: Name Marks):

```
Alice 95
Bob 80
Charlie 60
David 45
Eve 30
```

Step 3: Create GradeMapper.java

Command:

```
notepad GradeMapper.java
```

Paste this code:

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class GradeMapper extends Mapper<LongWritable, Text, Text, Text> {
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
    String[] data = value.toString().split(" ");
    String name = data[0];
    int marks = Integer.parseInt(data[1]);
    String grade;
    if (marks >= 90) grade = "A";
    else if (marks >= 75) grade = "B";
    else if (marks >= 50) grade = "C";
    else grade = "D";
    context.write(new Text(name), new Text(grade));
  }
}
```

Step 4: Create GradeReducer.java
Command:

```
notepad GradeReducer.java
```

Paste this code:

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class GradeReducer extends Reducer<Text, Text, Text, Text> {
  public void reduce(Text key, Iterable<Text> values, Context context)
      throws IOException, InterruptedException {
    for (Text val : values) {
      context.write(key, val);
    }
  }
}
```

Step 5: Create GradeDriver.java
Command:

```
notepad GradeDriver.java
```

Paste this code:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class GradeDriver {
  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.out.println("Usage: GradeDriver <input path> <output path>");
      System.exit(-1);
    }
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Student Grades");
```

```
    job.setJarByClass(GradeDriver.class);
    job.setMapperClass(GradeMapper.class);
    job.setReducerClass(GradeReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Step 6: Compile All Java Files
Command:

```
javac -classpath "%HADOOP_HOME%\share\hadoop\common\hadoop-common-
3.3.6.jar;%HADOOP_HOME%\share\hadoop\mapreduce\hadoop-mapreduce-client-core-
3.3.6.jar;%HADOOP_HOME%\share\hadoop\common\lib\commons-cli-1.2.jar" *.java
```

Step 7: Create JAR File
Command:

```
jar cf grades.jar *.class
```

Step 8: Upload Input File to HDFS
Commands:

```
hadoop fs -put C:\grades\student.txt /
hadoop fs -ls /
```

Step 9: Delete Old Output (if exists)
Command:

```
hadoop fs -rm -r /grades_output
```

Step 10: Run the MapReduce Job
Command:

```
hadoop jar grades.jar GradeDriver /student.txt /grades_output
```

Step 11: View the Output
Command:

```
hadoop fs -cat /grades_output/part-r-00000
```

Stopping Hadoop Services

Always stop Hadoop properly before closing CMD windows!

Step 1: Stop YARN

stop-yarn.cmd

Step 2: Stop HDFS

stop-dfs.cmd